

Relatório Atividade Prática 1 - INE5413

Alunos:

- Davi Ludvig - 23100473
- Pedro Henrique Gimenez - 23102766
- Victória Rodrigues - 23100460

1. Representação

- Para representar um grafo em memória, foi criada uma classe **Grafo** que contém um dicionário de vértices e outro de arestas. Cada vértice é representado por um objeto da classe **Vertice**, que contém um rótulo, uma lista vizinhos e um contador de grau. Cada aresta é representada por um par de rótulos de vértices.
- As funções solicitadas foram implementadas dentro da classe grafos, com uma ressalva.
 - **Grafo.qtdVertices()** -> **int**: retorna a quantidade de vértices do grafo.
 - Acessa a quantidade de elementos no dicinário de vértices.
 - **Grafo.qtdArestas()** -> **int**: retorna a quantidade de arestas do grafo.
 - Acessa a quantidade de elementos no dicionário de arestas.
 - **Grafo.grau(v: str)** -> **int**: retorna o grau do vértice passado como parâmetro.
 - Acessa o objeto Vertice armazenado no dicionário de vértices e retorna o atributo grau.
 - **Grafo.rotulo(v: str)** -> **str**: retorna o rótulo do vértice passado como parâmetro.
 - Acessa o objeto Vertice armazenado no dicionário de vértices e retorna o atributo rótulo.
 - **Grafo.vizinhos(v: str)** -> **List[str]**: retorna a lista de vizinhos do vértice passado como parâmetro.
 - Acessa o objeto Vertice armazenado no dicionário de vértices e retorna o atributo vizinhos.
 - **Grafo.haAresta(u: str, v: str)** -> **bool**: retorna True se existe uma aresta entre os vértices u e v, e False caso contrário.
 - Verifica se o vértice v está presente no atributo vizinhos do vértice u.
 - **Grafo.peso(u: str, v: str)** -> **float**: retorna o peso da aresta entre os vértices u e v.
 - Verifica se há aresta entre u e v e retorna o valor associado a ela armaenado no dicionário de arestas. Caso não há aresta, retorna **inf**
- Além destas, foram adicionadas as funções
 - **Grafo.cria_vertice(v: str)** -> **None**: cria um vértice com rótulo v.
 - Adiciona um objeto Vertice ao dicionário de vértices.
 - **Grafo.cria_aresta(u: str, v: str, peso: float)** -> **None**: cria uma aresta entre os vértices u e v com peso.
 - Checa se existem objetos Vertice de rótulo u e v no dicionário de vértices; caso não, cria-os.
 - Adiciona u ao atributo vizinhos de v e vice-versa.
 - Adiciona peso ao dicionário de arestas com chave (u, v) e (v, u).

- A função solicitada `ler(arquivo)` foi criada no arquivo `Auxiliar.py`, implementada na classe `Auxiliar`.
 - Acessa as linhas do arquivo `.net` e busca o número de vértices após a string `*vertices`.
 - Acessa as linhas das arestas que começam logo após a string `*edges`.
 - Cria vértices e arestas com os rótulos e pesos lidos, utilizando as funções `cria_vertice` e `cria_aresta` da classe `Grafo`.

2. [Buscas]

Para satisfazer as solicitações do enunciado, foi implementada a função `buscaEmLargura(grafo : Grafo, s : str)`.

- A fim de armazenar os dados necessários para a busca, foram criados os atributos
 - `Vertice.acessado` (bool): indica se o vértice foi acessado durante a busca. Inicia com False.
 - `Vertice.distancia` (float): indica a distância do vértice ao vértice de origem. Inicia com infinito.
 - `Vertice.anterior` (Vertice): indica o vértice anterior na busca. Inicia com None.
1. Iniciamos a BFS marcando o vértice de origem como acessado e com distância 0.
 2. Criamos uma fila e adicionamos o vértice de origem. Criamos um dicionário de níveis, que armazenará os vértices de cada nível - representado pela chave.
 3. Enquanto houver vértices na fila, retiramos o primeiro vértice e acessamos seus vizinhos.
 4. Se a distância do vizinho for nova (ainda não foi acessado nenhum vértice na mesma distância), cria-se um novo nível no dicionário de níveis.
 5. Adicionamos o vizinho à sua respectiva chave de nível e acessamos seus vizinhos.
 6. Para cada vértice vizinho não acessado, marcamos como acessado, definimos a distância e o vértice anterior, e adicionamos à fila.
 7. Enquanto houverem vizinhos a serem acessados, repetimos o processo.
 8. A saída esperada é o nível e seus vértices, em ordem de distância.

3. [Ciclo Euleriano]

Para satisfazer as solicitações do enunciado, foram implementadas as funções `hierholzer(grafo : Grafo)` e `buscarSubcicloEuleriano(grafo: Grafo, v: str, C: dict) -> tuple`.

1. Em `hierholzer(grafo : Grafo)`, inicializamos o dicionário C, onde cada chave é um vértice e seu valor começa com False.
2. Após isso, é selecionado um vértice arbitrário v.
3. v é utilizado para chamar a função `buscarSubcicloEuleriano(grafo: Grafo, v: str, C: dict) -> tuple`, que retorna r (booleano que indica se há um ciclo euleriano) e o ciclo euleriano.
4. Caso exista o ciclo, ele é exibido conforme o enunciado.
5. Em `buscarSubcicloEuleriano(grafo: Grafo, v: str, C: dict) -> tuple`, é criada uma lista de vértices chamada ciclo, que começa com v. Além disso, o valor de v é armazenado na variável t.
6. Repete-se o seguinte processo
 1. Se não existir nenhuma aresta saia de v que ainda não foi visitada, não existe ciclo (não existem arestas disponíveis e o fim do ciclo não é o vértice de origem [`v != t`]).

2. Caso contrário, para cada vértice u vizinho de v , se a aresta (v, u) não foi visitada, marcamos como visitada, adicionamos u ao ciclo e trocamos v por u .
7. Enquanto houverem arestas a serem visitadas e v for diferente de t , repetimos o processo.
8. Após isso, é necessário checar se existem algum ciclo não visitado no meio do ciclo principal.
9. Portanto, é acessado cada vértice x pertencente ao ciclo tal que este vértice possui alguma aresta não visitada.
10. Caso exista, é chamada a função `buscarSubcicloEuleriano(grafo: Grafo, x: str, C: dict) -> tuple` e o ciclo retornado é inserido no ciclo principal na posição de x .

4. [Algoritmo de Dijkstra]

Para satisfazer as solicitações do enunciado, foi implementada a função `dijkstra(grafo : Grafo, s : Vertice)` e a função `exibir(grafo : Grafo, s : Vertice)`, que formata a saída do algoritmo de Dijkstra como esperado.

- Esta solução utiliza a estrutura de dados `Heap mínimo`, que guarda o elemento de menor valor em sua raiz. O arquivo `min_heap.py` contém a implementação desta estrutura e foi enviado.
1. Inicializamos o algoritmo definindo a distância do vértice de origem como 0 e inicializando o `minHeap`.
 2. Insere-se o vértice de origem no `minHeap`.
 3. Num laço de inteiro até a quantidade de vértices, retiramos o vértice da raiz do `minHeap` e acessamos cada vizinho v .
 4. Caso a distância até v seja maior que a distância até u + o peso da aresta entre u e v , atualizamos a distância e o vértice anterior de v .
 5. Se v não foi acessado, inserimos v no `minHeap`.
 6. Este processo é repetido até que todos os vértices tenham sido acessados (ou seja, número de vértices vezes).

5. [Algoritmo de Floyd-Warshall]

Para satisfazer as solicitações do enunciado, foi implementada a função `floyd_warshall(grafo : Grafo) -> tuple` e a função `exibir(grafo : Grafo, d : list, p : list)`, que formata a saída do algoritmo de Floyd-Warshall como esperado.

1. São criadas as matrizes de distâncias e predecessores com altura e largura iguais ao número de vértices. Sendo d , toda preenchida com 0, a matriz de distâncias, e p , toda preenchida com None, a matriz de predecessores.
2. Para facilitar o acesso de rótulos de vértices, é criado um dicionário que mapeia rótulos a índices.
3. Para cada vértice u , acessamos novamente cada vértice v .
4. Caso $u=v$, a distância é 0 e o ancestral é nulo.
5. Caso contrário,
 1. Se há aresta entre u e v , a distância é o peso da aresta e o ancestral é u .
 2. Caso contrário, a distância é infinito e o ancestral é nulo.
6. Tendo estes dados preenchidos, é necessário acessar cada vértice k e verificar se a distância de u a v é menor que a distância de u a k + a distância de k a v .
7. Para cada vértice k , acessamos novamente cada vértice u e novamente cada vértice v .

8. Se a distância de u a v for maior que a distância de u a k + a distância de k a v , atualizamos a distância e o ancestral.
9. Retornamos as matrizes de distâncias e predecessores, que servem como entrada para a função de exibição.